



How Do Depth First and Breadth First Search Work?

As you start working with data structures and algorithms, you'll soon realize that one of the common operations that you'll need to perform is visiting each node.

This process is known as "traversing" the data structure.

Traversals are used to do something with every single node in the data structure, like printing their values, finding a specific value, or performing certain operations on the nodes.

By systematically visiting each node, you make sure that the process won't skip any nodes.

But how do you determine the order in which you should traverse the data structure? Where should the process start, and how should the next node be selected?

Without a clear way to traverse the data structure, going through it would be like walking through a maze without a specific path to follow.

That's where algorithms like breadth-first search (BFS) and depth-first search (DFS) become really important. They are commonly used to traverse graphs and for finding a path between two nodes.

When they are used to traverse a data structure, they define the order in which the nodes should be visited to make sure that none of them is skipped.

Let's start with breadth-first search (BFS).

Breadth-First Search (BFS)

Breadth-first search (BFS) is an algorithm that visits all neighboring nodes before moving to the next level in the graph.

It can be used to find the shortest path between two nodes in an unweighted graph because it analyzes all the nodes at each level, so it finds the path with fewest edges first.

This algorithm is commonly implemented using a queue data structure to keep track of the nodes that have been visited. Queues follow the FIFO (first in, first out) method, where the first node that was added to the queue is the first one to be removed.

The algorithm works like this:

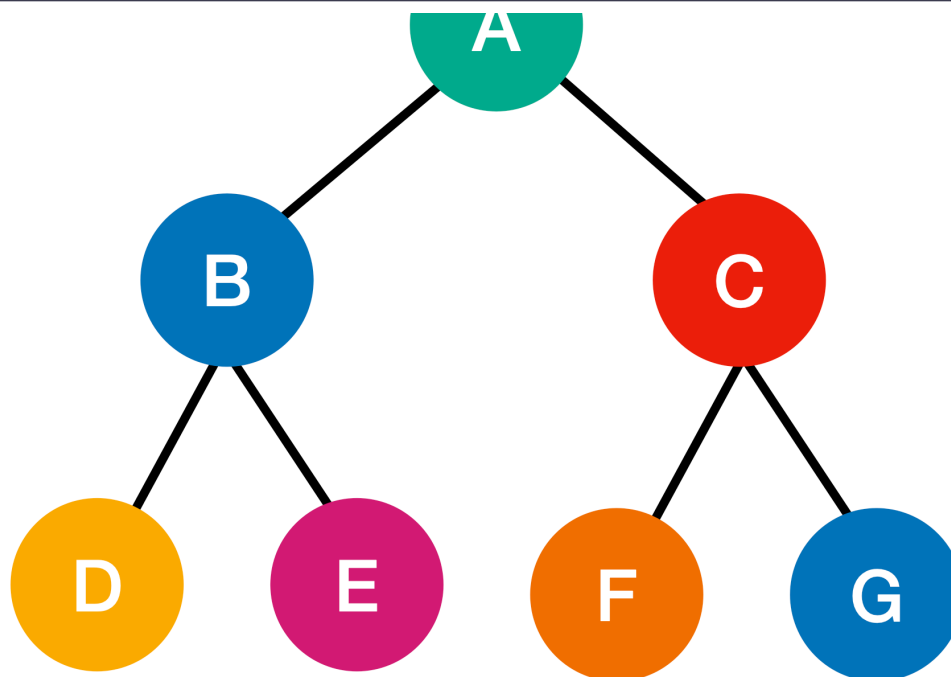
- You start at a specific node.
- That node is marked as visited and added to the queue.
- While the queue is not empty, the current node is removed from the queue (dequeued). Then, for each one of its neighbors, if the neighbor has not been visited, it is marked as visited and added to the queue.

One important consideration is that, since breadth-first search (BFS) requires storing a queue in memory, and this queue may have a large number of nodes, the space requirements of this algorithm can be considerable. This is especially true for graphs with a large number of nodes on the same level.

Let's see an example of BFS applied to a specific type of graph called a tree.

You will learn more about trees in an upcoming lesson, but they are essentially graphs with no cycles, where nodes are organized in a hierarchy. Cycles are paths that start and end at the same node.

Let's apply the breadth-first search (BFS) algorithm to this tree:

**Step 1:**

We start at the root of the tree, node A. We add A to the queue and immediately mark it as visited.

- Queue: [A]
- Visited: {A}

Step 2:

We dequeue node A. We add its unvisited children (node B and then node C), to the queue and mark them as visited.

- Queue: [B, C]
- Visited: {A, B, C}

The order in which nodes at the same level are added to the queue is defined by the implementation of the data structure and the order in which the edges (connections) are stored in the graph representation.

If the implementation is consistent, the specific order in which the nodes at the same level are traversed will not affect the correctness of the algorithm. It will still visit every node level by level.

Step 3:

We dequeue node B. We add its unvisited children, (node D and then node E), to the queue and mark them as visited.

- Queue: [C, D, E]
- Visited: {A, B, C, D, E}

Step 4:

We dequeue node C. We add its unvisited children, (node F and then node G), to the queue and mark them as visited.

- Queue: [D, E, F, G]
- Visited: {A, B, C, D, E, F, G}

Step 5:



- Visited: {A, B, C, D, E, F, G}

Step 6:

We dequeue node E. This node does not have any unvisited children, so nothing changes in the visited set.

- Queue: [F, G]
- Visited: {A, B, C, D, E, F, G}

Step 7:

We dequeue F. This node does not have any unvisited children, so nothing changes in the visited set.

- Queue: [G]
- Visited: {A, B, C, D, E, F, G}

Step 8:

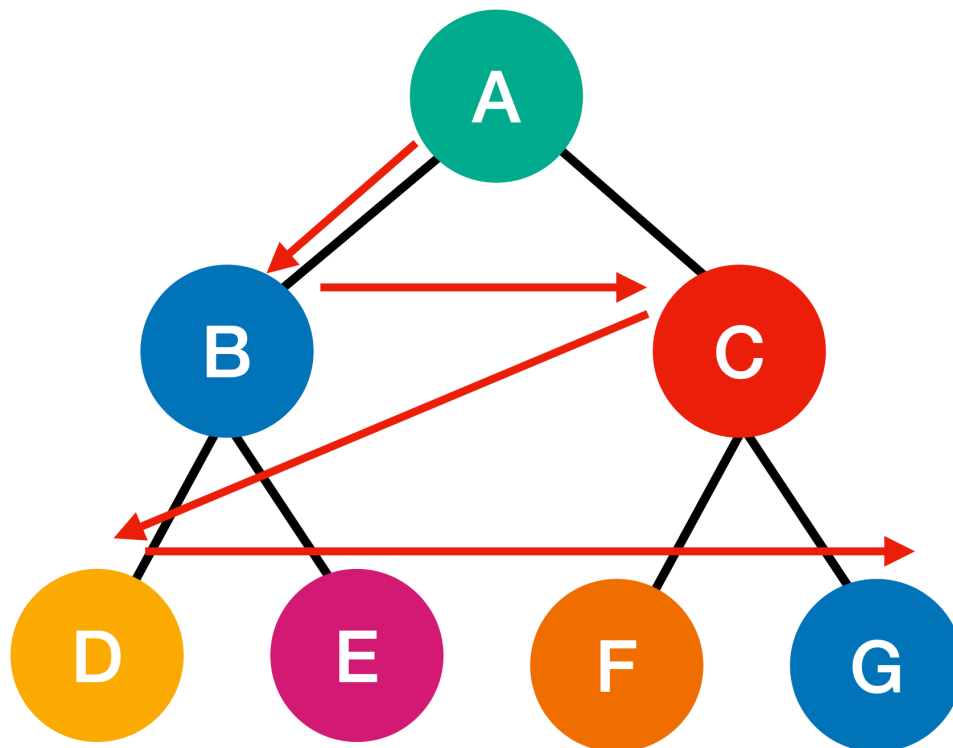
We dequeue G. This node does not have any unvisited children, so nothing changes in the visited set.

- Queue: []
- Visited: {A, B, C, D, E, F, G}

When the queue is empty, the traversal is complete.

The nodes were traversed in this order:

$A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow F \rightarrow G$



Notice how the algorithm visits the nodes per level.



Depth-First Search (DFS)

While breadth-first search (BFS) first visits all the neighboring nodes at the same level, **depth-first search (DFS)** follows each branch as deep as possible before it backtracks.

You can imagine this algorithm as exploring a maze by choosing a specific path and following it until you reach a dead end or the exit. If you reach a dead end, you go back and choose a different path.

Depth-first search (DFS) is commonly used to solve puzzles with a single solution, detecting cycles in a graph, and finding connected graph components.

This algorithm can be implemented using recursion or a stack data structure to keep track of the visited nodes.

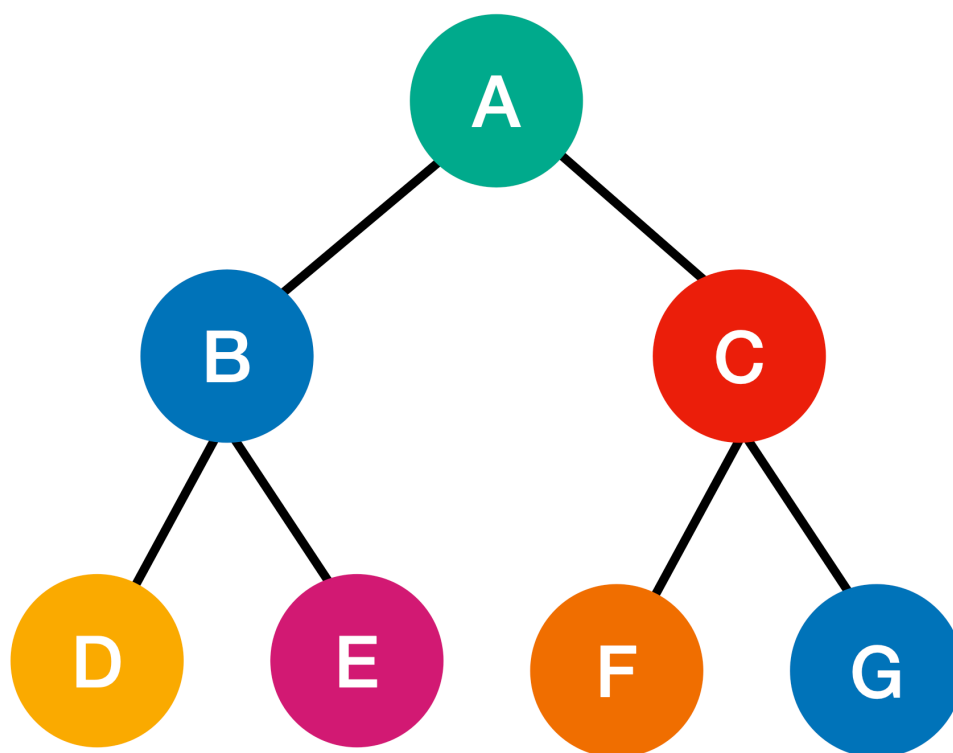
Stacks follow the LIFO (last in, first out) method, where the last node that was added to the stack is the first one to be removed from the stack.

The algorithm works like this:

- Start at a specific node.
- That node is marked as visited and added to the stack.
- While the stack is not empty, the current node is popped (removed). This is when we "visit" or process it (for example, by printing its value). Then, all of its unvisited neighbors are marked as visited and added to the stack.

One of the limitations of this algorithm is that it's not always guaranteed to find the shortest path between two nodes in an unweighted graph.

Let's see an example of Depth-First Search (DFS) applied to our tree example.



Step 1:

We start at the root node A. We mark it as visited and add it to the stack.



Step 2:

We pop node A from the stack.

Then, we add its unvisited children, node B and node C, to the stack. We'll add them in reverse order, C then B, so that B is on top (LIFO) and will be processed next. We also mark them as visited.

- Stack: [C, B]
- Visited: {A, B, C}

Step 3:

We pop node B from the stack.

Then, we add its unvisited children, node D and node E, to the stack in reverse order (E then D). We also mark them as visited.

- Stack: [C, E, D]
- Visited: {A, B, C, D, E}

Step 4:

We pop node D from the stack. This node has no children to add to the stack.

- Stack: [C, E]
- Visited: {A, B, C, D, E}

Step 5:

We pop node E from the stack. This node has no children to add to the stack.

- Stack: [C]
- Visited: {A, B, C, D, E}

Step 6:

We pop node C.

Then, we add its children, node F and node G, to the stack in reverse order (node G then node F) and we mark them as visited.

- Stack: [G, F]
- Visited: {A, B, C, D, E, F, G}

Step 7:

We pop node F from the stack. This node has no children to add to the stack.

- Stack: [G]
- Visited: {A, B, C, D, E, F, G}

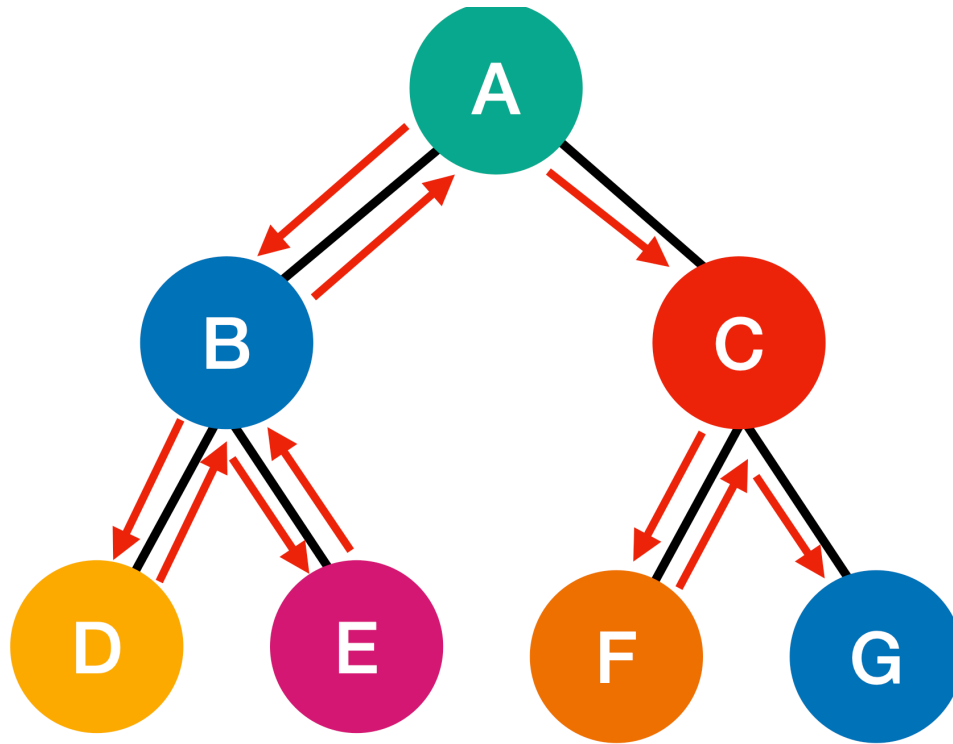
Step 8:

We pop node G. This node has no children to add to the stack.

- Stack: []
- Visited: {A, B, C, D, E, F, G}

When the stack is empty, the traversal is completed and all nodes have been visited.

The algorithm visited the nodes in this order:



Notice how we start at node A, and then move all the way down the tree to node B, and nodes D and E, before we move up again to node C and then nodes F and G. This is the core principle of depth-first search (DFS), traversing full paths before backtracking and finding other paths.

In this case, we solved this example using a stack. Alternatively, depth-first search (DFS) can be implemented using recursion, where the function processes the current node and then calls itself for each of its unvisited neighbors. The function call stack implicitly manages the LIFO (last-in, first-out) order.

Both breadth-first search (BFS) and depth-first search (DFS) are essential algorithms for traversing graphs and trees. Breadth-first search (BFS) explores nodes level by level, which is perfect for finding the shortest path in an unweighted graph. On the other hand, depth-first search (DFS) follows one branch as deep as possible before backtracking, which is perfect for solving mazes and detecting cycles. Understanding their pros and cons is helpful for choosing the right one for a particular problem.

Questions

Which of the following data structures is commonly used to implement a standard breadth-first search (BFS) algorithm?

Stack
Queue
Correct!
Linked List



Python Certification



Understanding Graphs and Trees

Which of the following statements about depth-first search (DFS) is true?

Depth-first search is guaranteed to find the shortest path between two nodes in an unweighted graph.

Depth-first search visits all neighbors at the current level before moving to the next level.

Depth-first search is always more space-efficient than BFS.

Depth-first search is typically implemented using recursion or a stack.

Correct!

If you wanted to find the shortest path from a starting node to a target node in an unweighted graph, which algorithm would be the most suitable choice?

Breadth-first search (BFS)

Correct!

Depth-first search (DFS)

Binary search

Merge sort

Check your answer

Ask for Help